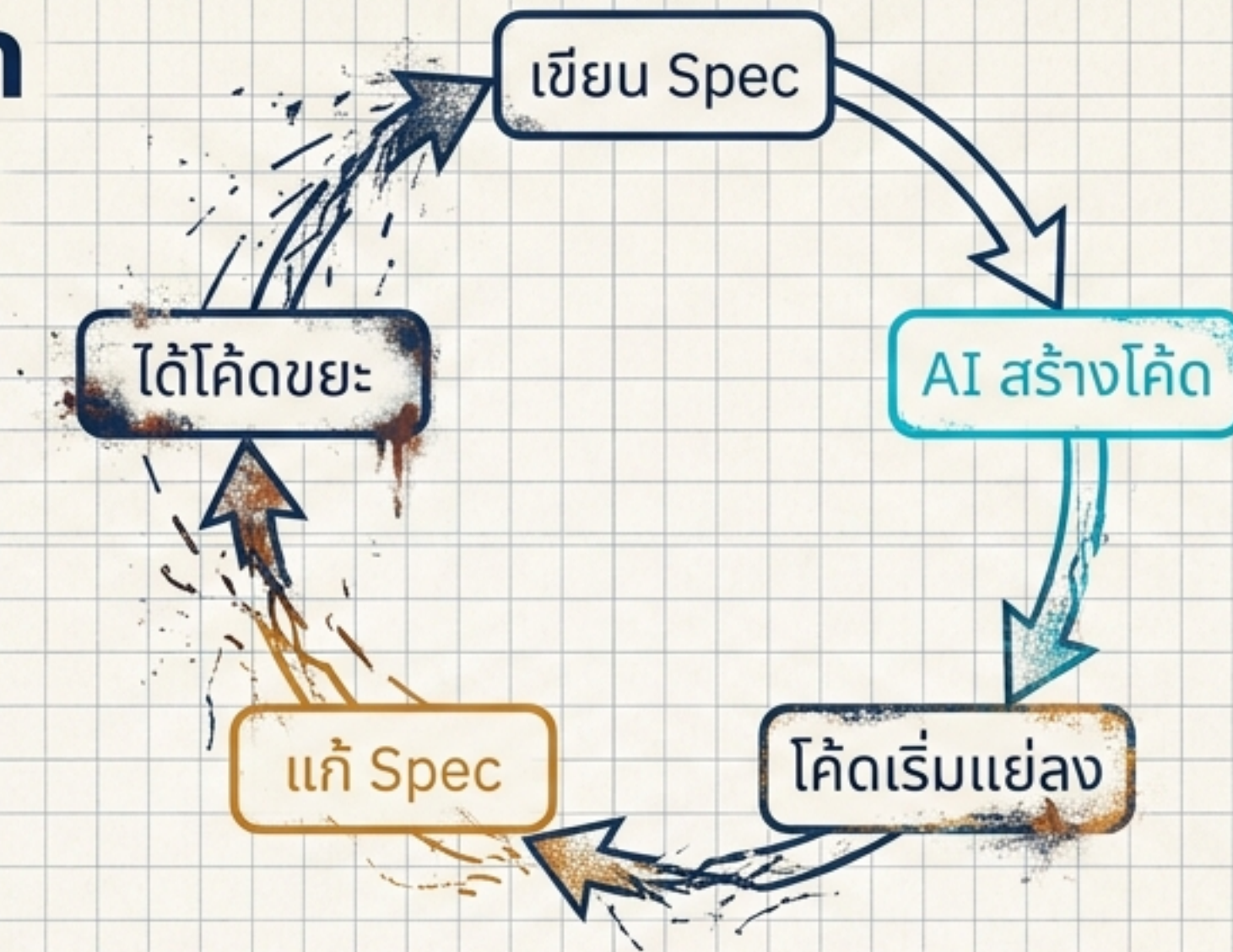


กับดักของแนวคิด Specs-to-Code



ความเชื่อ:

โค้ดจะจัดการตัวเองได้
แค่แก้ไข Spec แล้วรันคอมไพล์ใหม่



SELF_COMPILING_IDEAL

ความจริง:

ระบบจะเข้าสู่สภาวะ Software Entropy
(ความไร้ระเบียบ)



SOFTWARE_ENTROPY_STATE

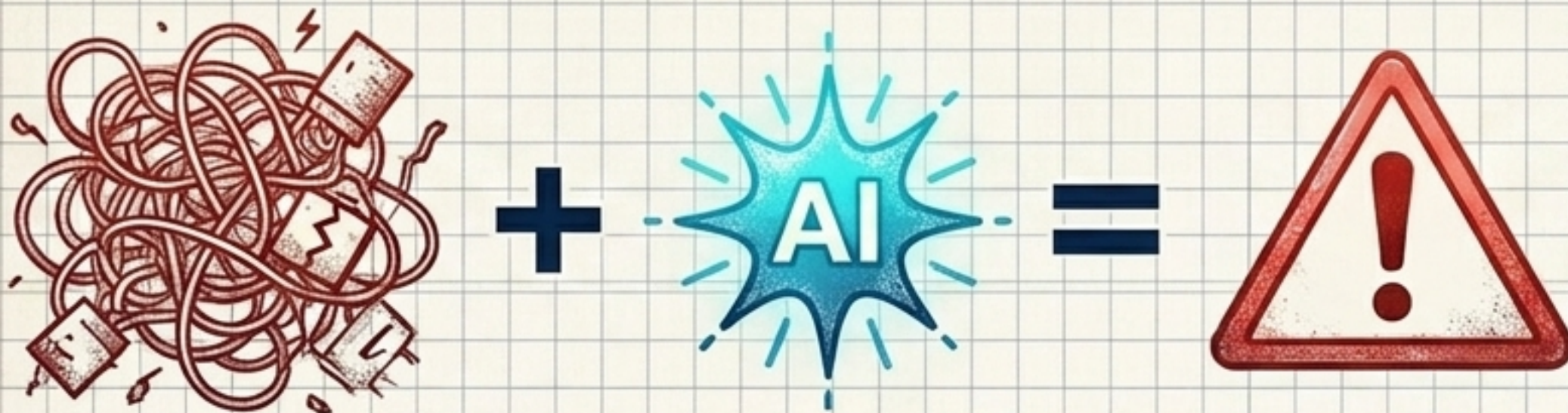
ผลลัพธ์:

การแก้อจุดหนึ่งนำไปสู่ความพังพินาศในอีกจุด
โค้ดที่ได้จะแย่งเรื่อยๆ

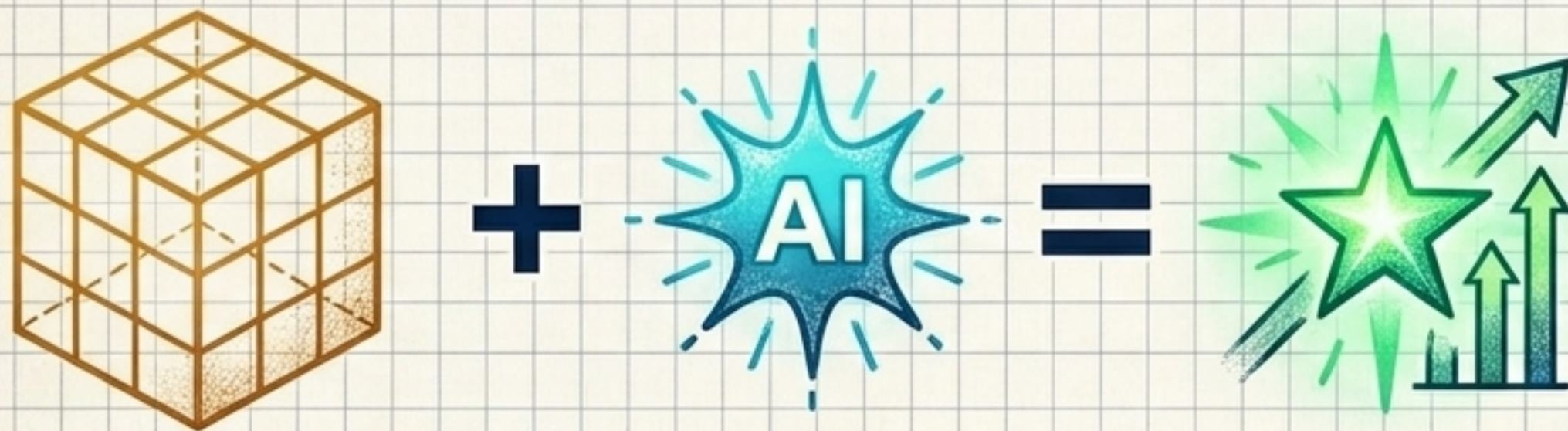


CASCADING_FAILURE_EFFECT

โค้ดที่แย่ในยุค AI มีราคาแพงที่สุด



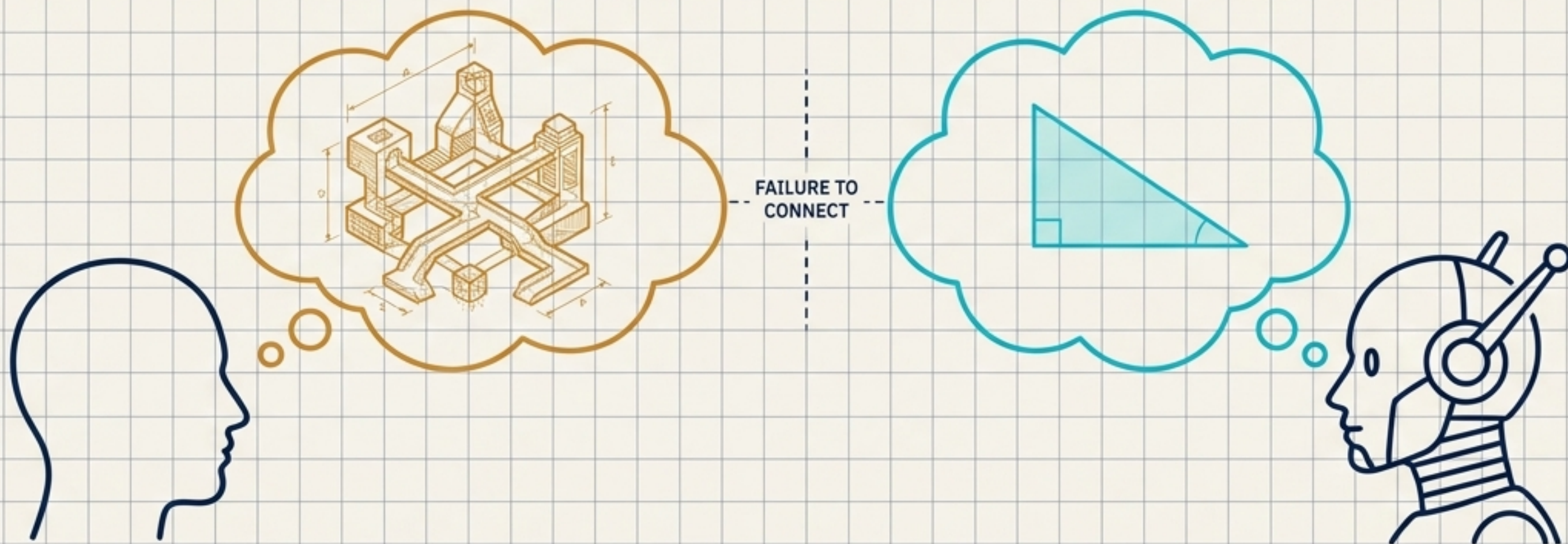
โค้ดเบสที่แก้ไขยาก + AI
= ความพังพินาศที่มีราคาแพง



โค้ดเบสที่ดี + AI
= ขุมทรัพย์แห่งการต่อยอดไร้ขีดจำกัด

AI จะทำงานได้ยอดเยี่ยมที่สุด เมื่ออยู่ในโค้ดเบสที่มีพื้นฐานการออกแบบที่ดีเยี่ยมเท่านั้น

AI สร้างสิ่งที่ไม่ตรงกับความต้องการของคุณ



สาเหตุ: ขาดสิ่งที่เรียกว่า Design Concept (อ้างอิงจาก The Design of Design - Frederick P. Brooks)

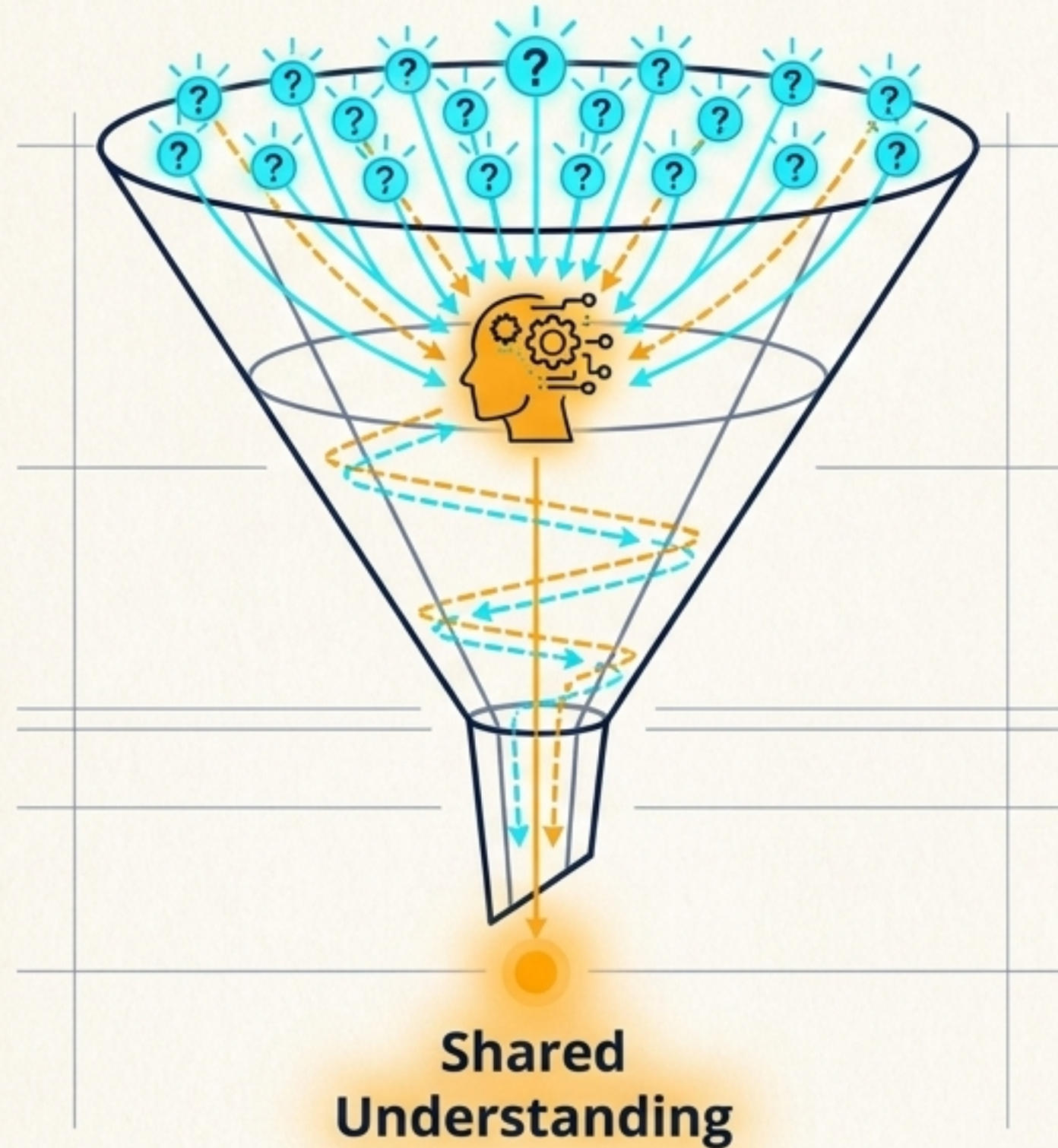


ปัญหา: Design Concept ไม่ใช่แค่ไฟล์ Markdown แต่เป็น ทฤษฎีและภาพรวมของระบบที่ ลอยอยู่ระหว่างคนสองคน



พฤติกรรม AI: เครื่องมือ AI มักจะอยู่ในโหมด อยากเขียน แผนงาน (Eager Plan Mode) จนข้ามขั้นตอนการทำความเข้าใจ

ใช้เทคนิค Grill Me เพื่อสร้างความเข้าใจที่ตรงกัน



Interview me relentlessly about every aspect of this plan until we reach a shared understanding.

Walk down each branch of the design tree, resolving dependencies between decisions one by one.

1. บังคับให้ AI ถามคำถาม 40-100 ข้อ
2. เปลี่ยน AI จาก ผู้รับคำสั่ง เป็น ผู้ร่วมตรวจสอบ (Adversary)
3. ยืนยันโครงสร้างจนกว่า Design Concept จะตรงกันทั้งหมด

ทลายกำแพงภาษาด้วย Ubiquitous Language



Domain-Driven Design

ปัญหา:



AI ใช้คำฟุ่มเฟือย อธิบายวกวน
หรือใช้คำศัพท์ที่ไม่ตรงกับบริบทของธุรกิจ

ทางออก:



สร้างไฟล์ Markdown รวมคำศัพท์เฉพาะ
(Domain Model) ที่ทุกคน (และ AI) ต้องใช้ตรงกัน

ผลลัพธ์:



AI ใช้คำน้อยลง แต่มีความแม่นยำในการเขียนโค้ด
และวางแผนสูงขึ้นอย่างเห็นได้ชัด

ความเร็วที่เกินขีดจำกัดของการตรวจสอบ



- **อาการ:** AI เขียนโค้ดปริมาณมหาศาลรวดเร็วแล้วค่อยคิดได้ทีหลังว่าควร**ตรวจเช็ค** Type หรือรับ Test
- **หลักการ:** Outrunning your headlights (ขับรถเร็วกว่าระยะที่ไฟหน้าส่องถึง)
- **กฎเหล็ก:** ขีดจำกัดความเร็วในการเขียนโค้ดของคุณ ถูกกำหนดโดยความเร็วของ Feedback Loop ในระบบ

TDD คือตัวควบคุมความเร็วของ AI

1. เขียน Test ก่อน:

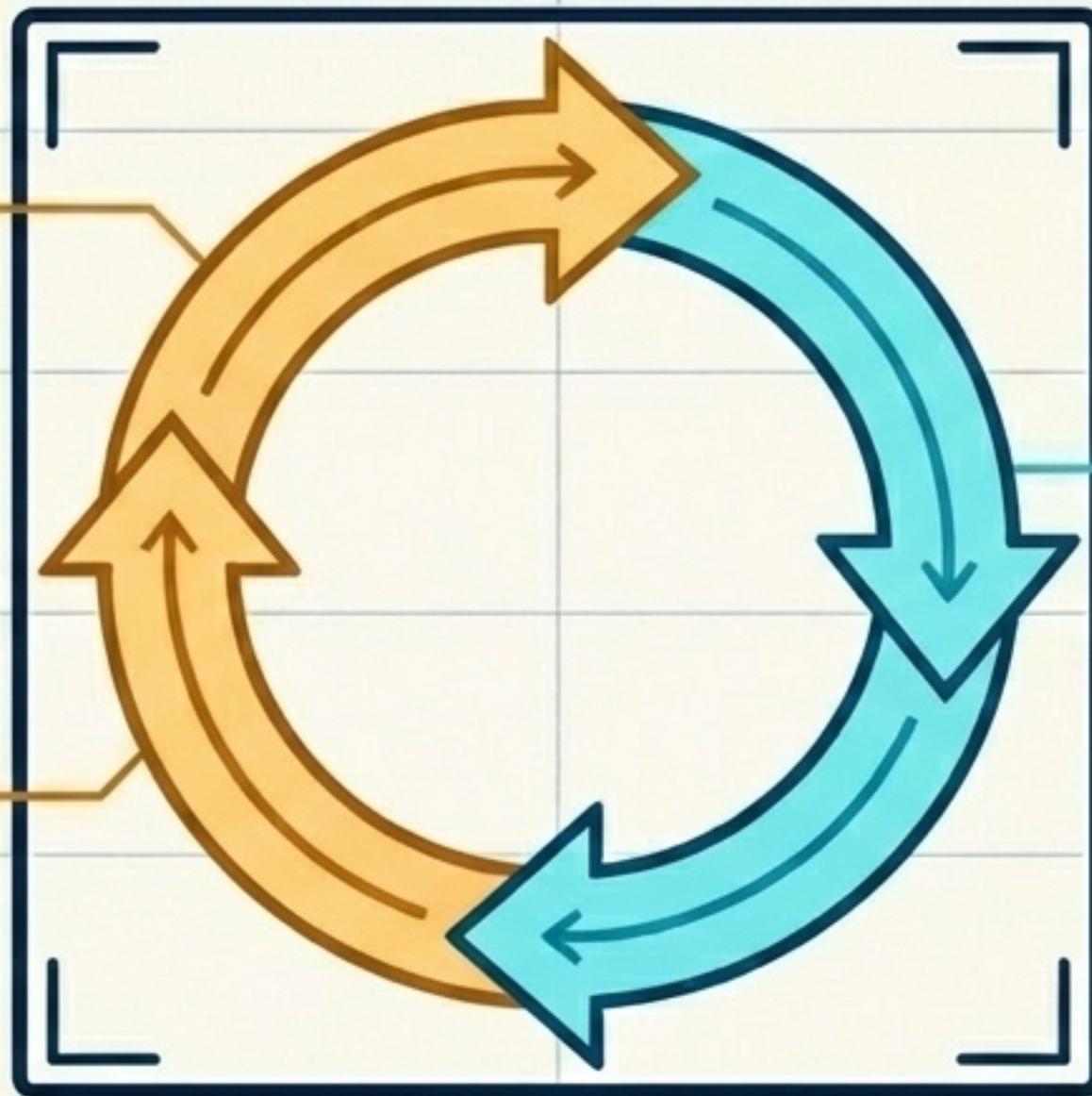
บังคับให้กำหนด
พฤติกรรมที่ต้องการ

3. ตรวจสอบและปรับปรุง:

Refactor โค้ดให้สวยงาม
โดยคำนึงถึงการออกแบบ

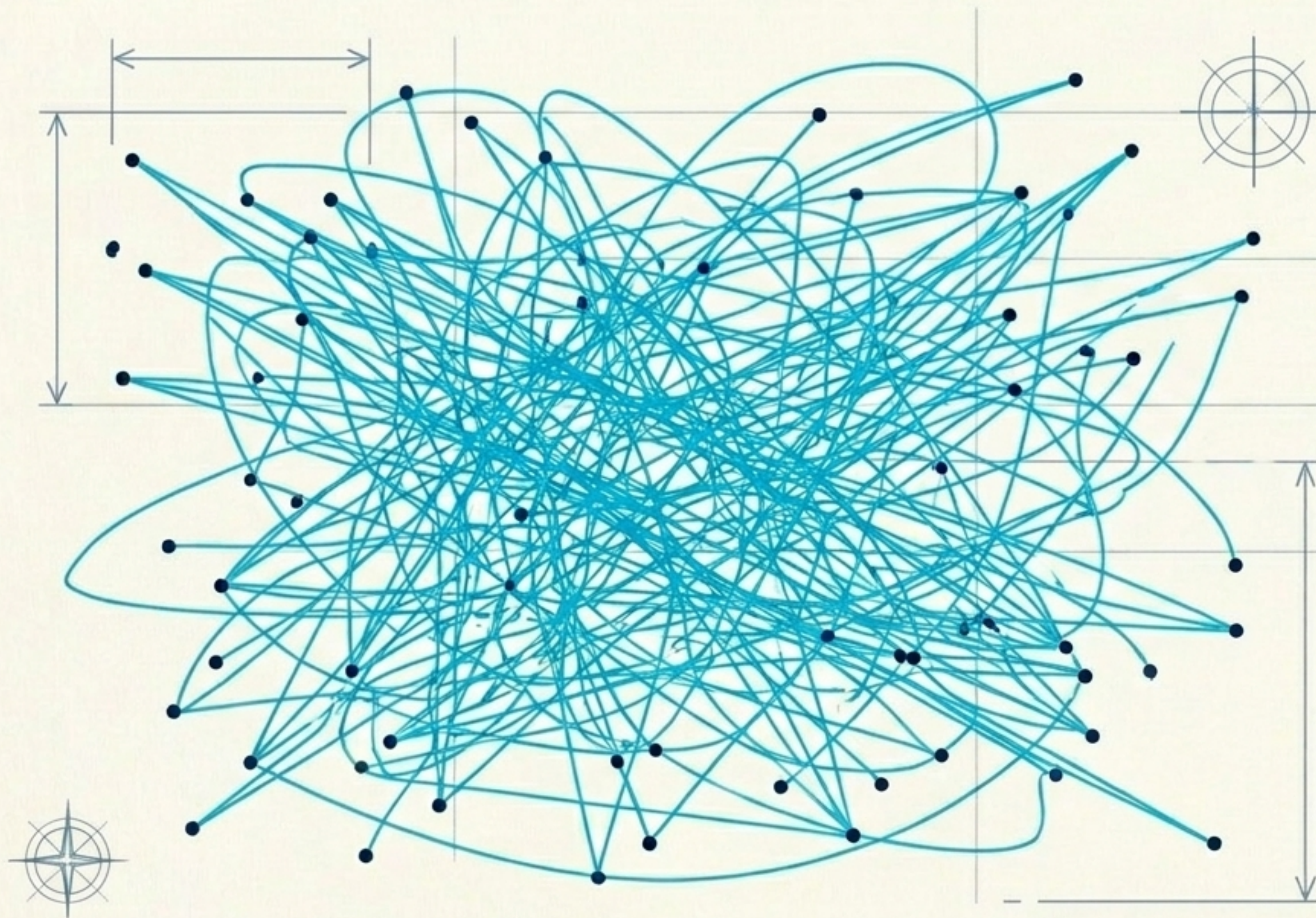
2. AI เริ่มทำงานทีละนิด:

ขีดกรอบให้ AI เขียนโค้ดสั้นๆ
แค่ให้ผ่าน Test



Takeaway: TDD บังคับให้ AI ก้าวเดินทีละก้าวอย่างมีสติ
แทนการวิ่งพุ่งชนกำแพง

AI หลงทางในโค้ดเบสที่ซับซ้อนและกระจาย



ข้อมูลและลอจิกถูกสับย่อยเป็น
ชิ้นเล็กชิ้นน้อย (Tiny blobs)



ฟังก์ชันมีน้อย แต่ Interface
โยงกันซับซ้อนยุ่งเหยิง



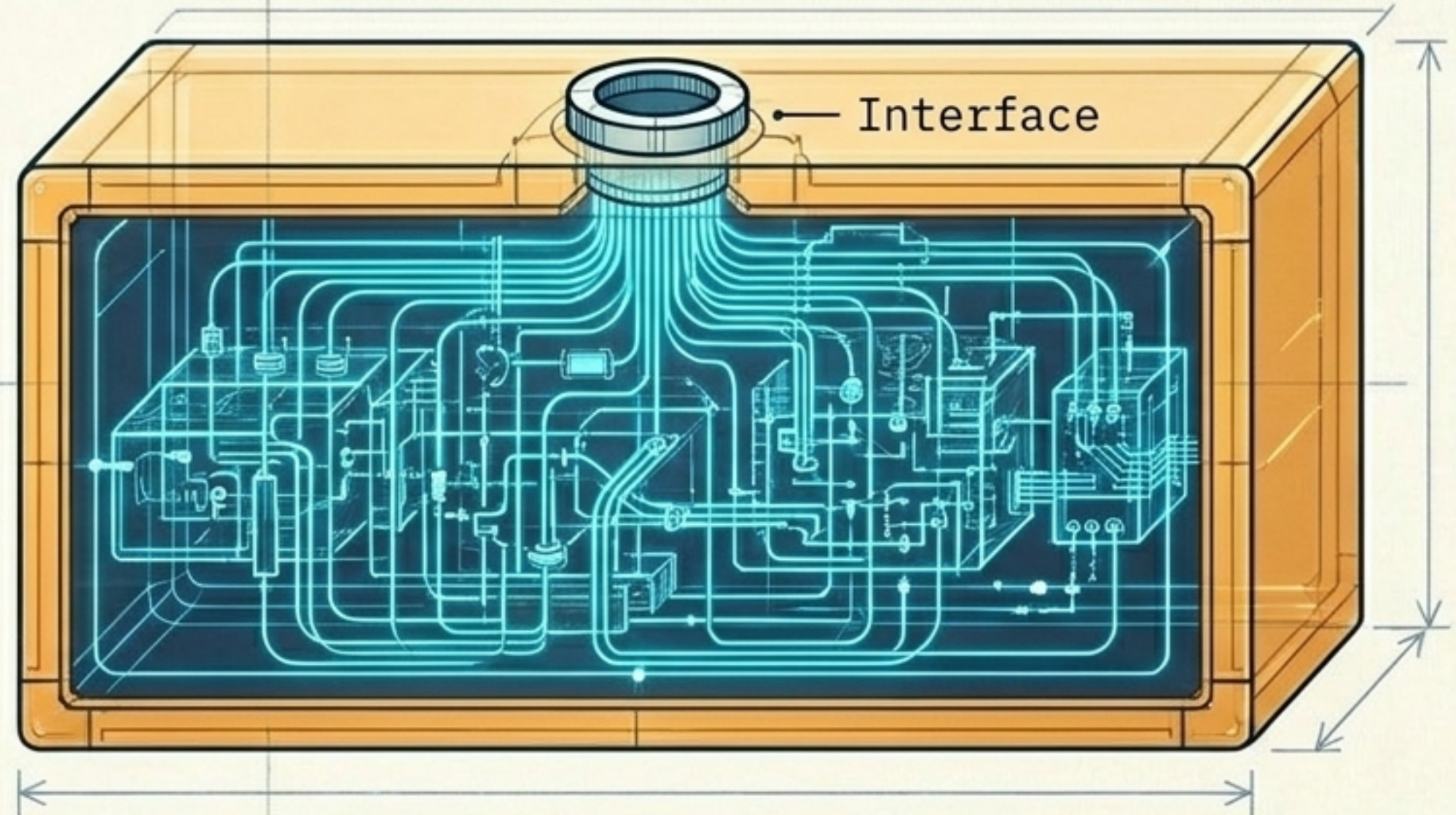
AI ไม่สามารถทำความเข้าใจ
ภาพรวมได้ นำไปสู่การเรียกใช้
ฟังก์ชันผิดพลาดและทำลาย
Dependency ดั้งเดิม

สร้าง Deep Modules เพื่อตีกรอบและนำทาง AI

Shallow Modules



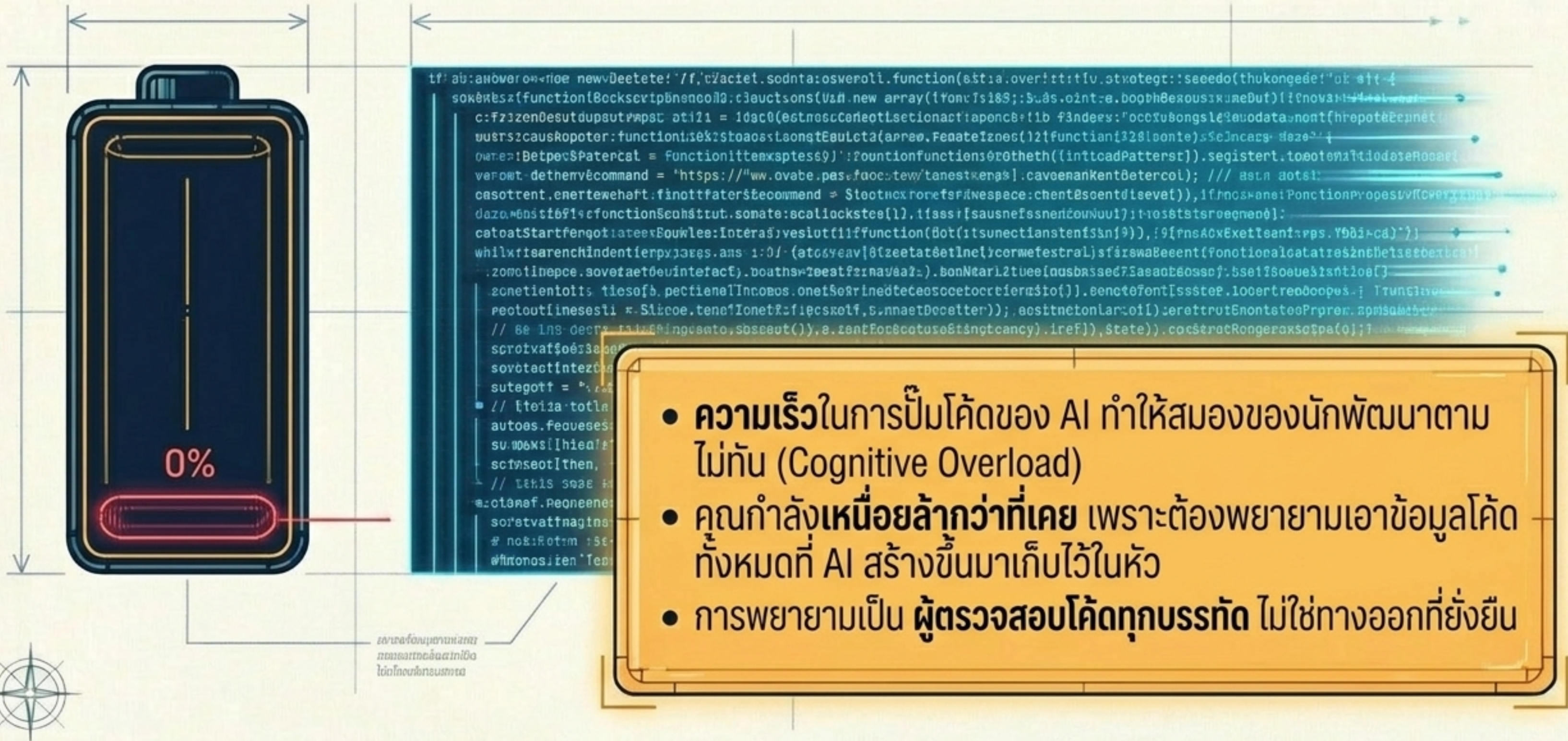
Deep Module



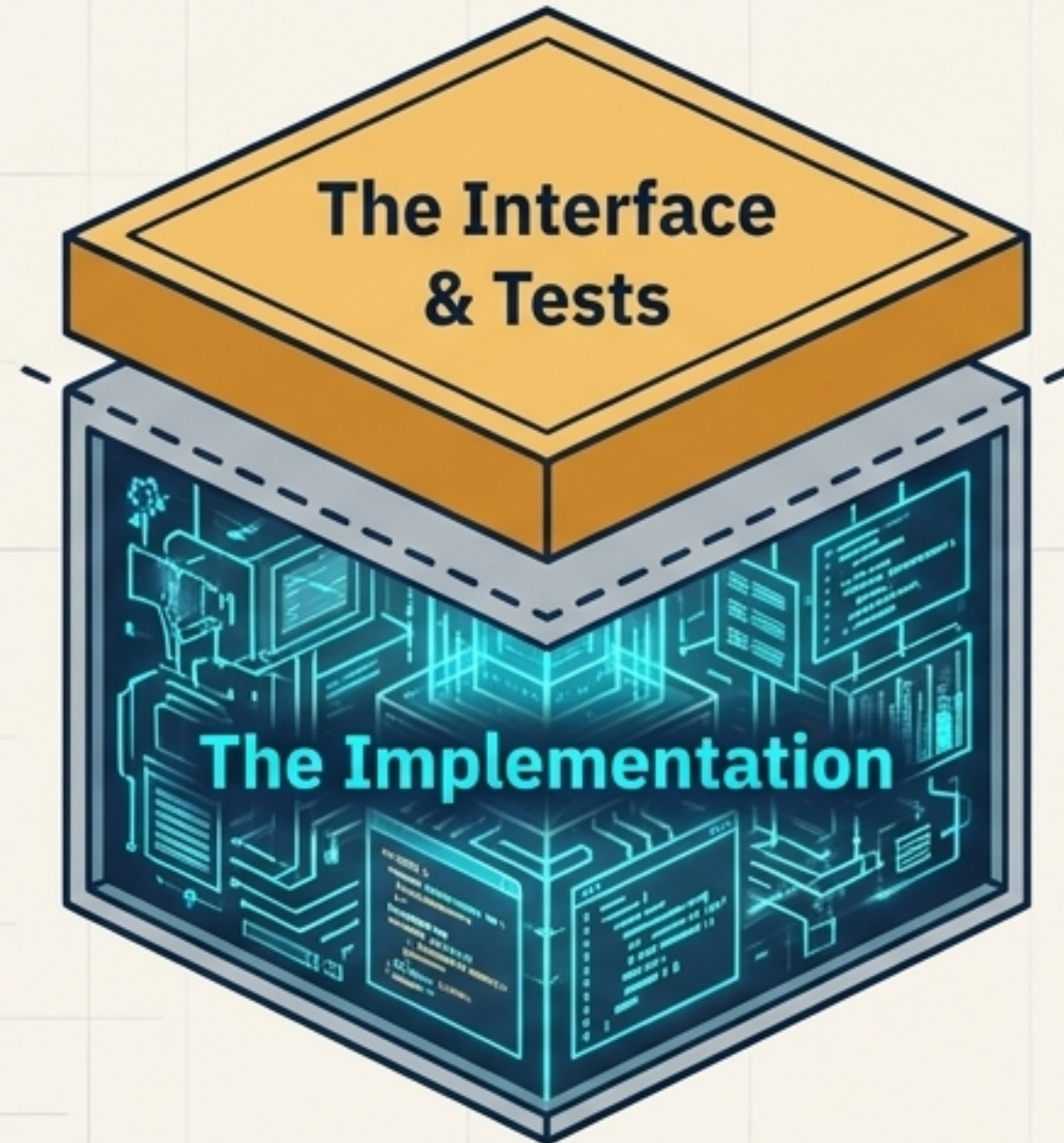
- **ซ่อนความซับซ้อน:** ลอจิกจำนวนมหาศาลถูกซ่อนอยู่หลัง Interface ที่เรียบง่ายเพียงจุดเดียว
- **ทดสอบง่าย:** กำหนดขอบเขตชัดเจน ทำให้ง่ายต่อการเขียน TDD
- **สวรรค์ของ AI:** AI สามารถโฟกัสที่การทำ Implementation ภายในกล่องได้โดยไม่กระทบกับระบบอื่น



ภาระทางความคิดเมื่อต้องไล่อ่านโค้ดของ AI



ออกแบบ Interface แล้วปล่อย Implementation ให้ AI



มนุษย์ (Amber):

- ไฟล์ที่การออกแบบ Interface ภายนอก
- ทำความเข้าใจจุดประสงค์ และเขียน Test กลุ่มขอบเขตเอาไว้

AI (Cyan):

- ปล่อยให้ AI จัดการรายละเอียด Implementation ที่อยู่ภายในกล่อง (Gray Box)

ผลลัพธ์: ลดการระดมสมอง คุณไม่ต้องรู้ทุกบรรทัด ตราบใดที่ Interface ยืนยันผ่าน Test

บทบาทใหม่ สถาปนิกผู้กำหนดทิศทางระบบ



“Invest in the design of the system every day.”

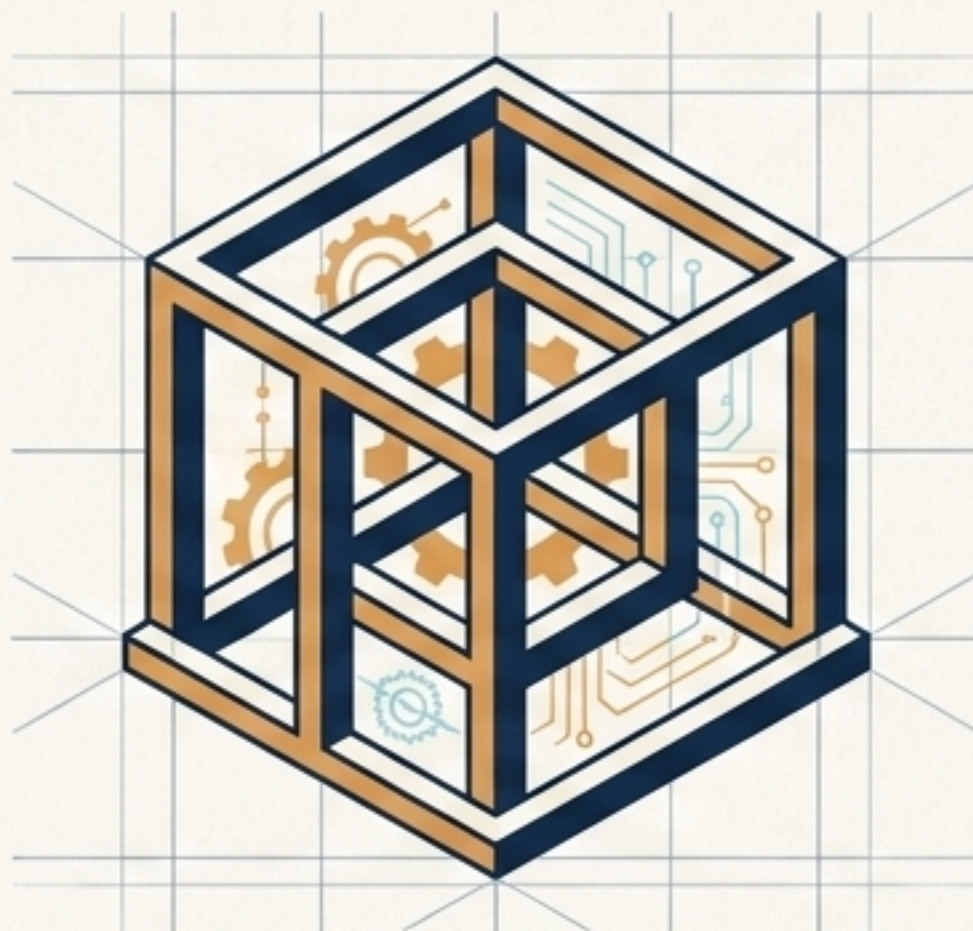
- Kent Beck

Context: AI คือทหารราบที่เก่งกาจในการลงมือทำ แต่ระบบยังต้องการนายพลที่มองเห็นภาพรวมเชิงกลยุทธ์เหนือสมรภูมิ

สรุปกระบวนการทัศน์ Builder ทั่วไป vs. AI Architect

แนวคิด Specs-to-Code (อดีต)	แนวคิด AI-Assisted Architecture (ปัจจุบัน)
มุมมองต่อโค้ด: โค้ดเป็นของราคาถูก โยนทิ้งได้	มุมมองต่อโค้ด: โค้ดคือสินทรัพย์ การออกแบบที่ดีมีมูลค่ามหาศาล
สถานะของ AI: เป็นเครื่องพิมพ์ดีดวิเศษ	สถานะของ AI: เป็นผู้ร่วมงานเชิงยุทธวิธี (Sergeant on the ground)
การจัดการ AI: รับคอมไพล์ซ้ำๆ หวังผลลัพธ์ใหม่	การจัดการ AI: ใช้ TDD และ Grill Me เพื่อตีกรอบที่ละก้าว
โครงสร้างระบบ: Shallow Modules กระจายกระจาย	โครงสร้างระบบ: Deep Modules ช้อนความซับซ้อน

ทักษะวิศวกรรมซอฟต์แวร์ของคุณมีค่ามากกว่าที่เคย



แนวคิดแบบ Specs-to-Code คือการละทิ้งการออกแบบ แต่ในยุคนี้ การลงทุนกับพื้นฐานสถาปัตยกรรมระบบคือวิธีเดียวที่จะปลดล็อกพลังที่แท้จริงของ AI

```
>_ ฝึกฝนทักษะ AI Skills Framework: github.com/mattpocock/skills  
>_ ติดตามบทความเพิ่มเติม: aihero.dev
```